

Meta-aprendizagem

Automação de workflow / design de pipelines

Prof. Lucas Amorim - lucas@ic.ufal.br

1

Introdução

- Em muitas situações práticas, o objetivo não é apenas selecionar um algoritmo e suas configurações.
- Ao invés disso busca-se definir toda uma sequência de algoritmos necessários para realizar uma tarefa de aprendizagem de máquina.
- Discutiremos o design de workflows (ou pipelines) que representam soluções que envolvem mais de um algoritmo.
- Tipicamente a sequência se inicia com transformações dos dados e finaliza com um algoritmo de aprendizado de máquina.

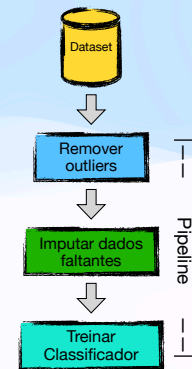


Fig.: Exemplo de pipeline.

Ref

2

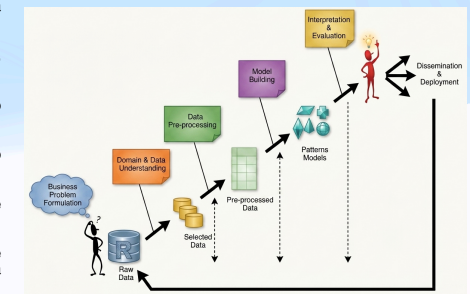
Sumário

O processo de KDD Definição do espaço de busca Estratégias usadas em design de pipelines MetaML

3

O processo de KDD

- KDD - Knowledge Discovery from Data.
- Conhecimento é o produto final da descoberta orientada a dados.
- Às vezes Data Mining (DM) é usado como sinônimo de KDD, às vezes é entendido como parte do processo de KDD.
- Aprendizagem de máquina é aplicada dentro de um processo de KDD.
- Algumas etapas de KDD requerem maior participação humana.
- O estágio inicial e final requerem bastante conhecimento de domínio.
- As etapas mais algorítmicas (pre-processamento dos dados e construção do modelo) são candidatas ideais para automação.
- Esta automação pode ocorrer através do uso de **meta-conhecimento**.



Ref

4

Definição do espaço de busca



- Sistemas de mineração de dados ou mesmo de AutoML precisam levar em conta duas importantes etapas:
 - Definição do espaço de pipelines alternativos (espaço de configurações);
 - Busca neste espaço pelo melhor pipeline para o dataset alvo.
- Detalharemos em seguida como essas duas etapas vem sendo implementadas.

5

Definição do espaço de busca



- No contexto de definição de pipelines, a definição do espaço de busca envolve duas questões:
 - Definição dos constituintes básicos (operadores) do pipeline;
 - Definição das maneiras como estes constituintes podem ser combinados para gerar os pipelines.
- Alguns pesquisadores introduziram a noção de portfólios de algoritmos para representar diferentes conjuntos de algoritmos ou pipelines (Gomes et al. 2001).

Gomes, C. P. and Selmany, B. (2001). Algorithm portfolios. Artificial Intelligence, 126(12):43–62.

6

Definição do espaço de busca



- Outra abordagem é o uso de ontologias, para agrupar algoritmos semelhantes.
- Ontologias (Chandrasekaran e Jopheson, 1999) permitem descrever um conjunto de constituintes de pipelines, os operadores.
- Elas foram empregadas em vários sistemas de ML e DM, a exemplo de:
 - DM ontologia usada no IDA (Bernstein and Provost, 2001)
 - Ontologia baseada em OWL-DL usada no sistema RDF (Patel-Schneider et al., 2004)
 - DMWF, Data Mining Workflow Ontology usado no sistema eIDA (Kietz et al., 2009)
 - KDDONTO (Diamantini et al., 2012)
 - DMOP (Hilario et al., 2009)
 - KD ontology (Zakova et al., 2011)
 - Exposé ontology (Vanschoren et al., 2012)
 - Auto-Weka parameter space (Thornton et al., 2013)

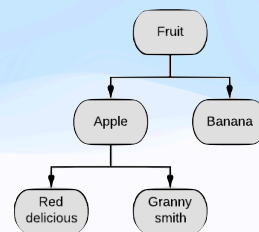


Fig.: Exemplo de Ontologia

Chandrasekaran, B. and Jopheson, J. (1999). What are ontologies, and why do we need them? IEEE Intelligent Systems, 14(1):20–26

7

Definição do espaço de busca



IDA

- Uma ontologia simplificada foi usada no sistema IDA de Bernstein e Provost (2001).
- A camada inferior contém os **operadores concretos**, tais como random sampling (rs), fixed-bin discretization (fbd), class-based discretization (cbd), CPE-thresholding post-processor.
- A camada acima mostra o que alguns chamam de operadores abstratos: pré-processamento, construção de modelo, pós-processamento.
- Um exemplo de pipeline resultante dessa ontologia: fbd; nb; cpe.

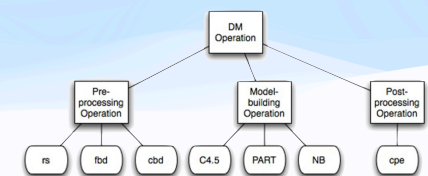


Fig.: Ontologia usada no sistema IDA.

Bernstein, A. and Provost, F. (2001). An intelligent assistant for the knowledge discovery process. In Hsu, W., Kargupta, H., Liu, H., and Street, N., editors, Proceedings of the IJCAI-01 Workshop on Wrappers for Performance Enhancement in KDD.

8

Definição do espaço de busca



Auto-WEKA

- A figura mostra uma ontologia usada para representar o espaço de parâmetros do sistema Auto-WEKA (Thornton et al., 2013).
- A parte superior lida com a seleção de métodos de classificação e seus parâmetros.
- A parte inferior da figura lida com seleção/avaliação de features.

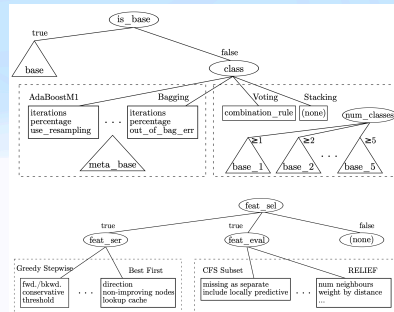


Fig.: Ontologia do espaço de parâmetros usado no Auto-WEKA.

Thornton, C., Hutter, F., Hoos, H. H., and Leyton-Brown, K. (2013). Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. In Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pages 847–855. ACM.

9

Definição do espaço de busca



- Ontologias não especificam a ordem em que as operações devem ser aplicadas, nem se todos ou somente alguns dos operadores de cada ramo precisam ser aplicados.
- Ontologias podem resultar em pipelines inválidos como “cpe; nb; rs”.
- Precisamos de maneiras para determinar a ordem em que o espaço de busca precisa ser buscado. O que se chama de **viés procedimental**.
- Diferentes maneiras de impor um viés procedimental:
 - Heuristic rankers;
 - Gramáticas livres de contexto;
 - Operadores com pre-condições (requisitos) e pós-condições (efeitos).

10

Definição do espaço de busca



Heuristic ranker

- Um heuristic ranker (ordenador heurístico) foi utilizado no IDA.
- Ele ordena as operações válidas usando uma função heurística que incorpora as preferências do usuário sobre velocidade, acurácia e interpretabilidade do modelo.
- Essas características são especificadas na ontologia para todos os operadores.
- Um problema com essa abordagem é que essas características são dependentes de cada caso individualmente e requerem experimentação.

11

Definição do espaço de busca



Gramáticas livres de contexto (CFGs):

- O formalismo das CFGs nos permite não somente definir o espaço de modelos alternativos mas também impor restrições sobre o processo de busca.
- Uma CFG é uma quádrupla (S, V, Σ, R) , onde S é o símbolo inicial, V é um conjunto de símbolos não terminais, Σ um conjunto de símbolos terminais, e R um conjunto de regras de produção especificando a substituição dos símbolos (Hopcroft e Ullman, 1979).
- Símbolos não-terminais podem ser substituídos, os terminais não.
- Substituições válidas são expressas pelas regras em R .

```

1.  $S \leftarrow DM.Oper$ 
2.  $DM.Oper \leftarrow Pre.Proc ; Model.Build$ 
3.  $Pre.Proc \leftarrow Sampling ; Discret$ 
4.  $Sampling \leftarrow rs \mid null$ 
5.  $Discret \leftarrow fbd \mid cbd \mid null$ 
6.  $Model.Build \leftarrow Cat.Classif \mid Prob.Classif.Post$ 
7.  $Cat.Classif \leftarrow c4.5 \mid part$ 
8.  $Prob.Classif.Post \leftarrow nb ; cpe$ 
    
```

Fig.: Exemplo de CFG para construção de pipelines no sistema IDA.

Nº	Workflow
1	c4.5
2	part
3	nb; cpe
4	rs; c4.5
5	rs; part
6	rs; nb; cpe
7	fbd; c4.5
8	fbd; part
9	fbd; nb; cpe
10	cbd; c4.5
11	cbd; part
12	cbd; nb; cpe
13	rs; fbd; c4.5
14	rs; fbd; part
15	rs; fbd; nb; cpe
16	rs; cbd; c4.5
17	rs; cbd; part
18	rs; cbd; nb; cpe

Fig.: Exemplos de pipelines gerados pelo IDA.

Hopcroft, J. E. and Ullman, J. D. (1979). Introduction to Automata Theory, Languages, and Computation. Addison-Wesley

12

Definição do espaço de busca



Indução de CFGs a partir de exemplos:

- Pode ser conveniente prover uma lista de pipelines válidos e inválidos ao invés de um procedimento formal ou uma gramática que possa ser executada.
- Vários pesquisadores seguiram esse caminho de indução de gramática (ex.: Duda et al., 2001).
- A ideia principal é encontrar padrões frequentes, como “nb; cpe”, ou “ss; knn” nos exemplos. Os padrões mais frequentes podem ser substituídos por um novo símbolo (integrados).
- Um problema com essa abordagem é que os novos símbolos gerados podem não se adequar aos dados do usuário.
- Além disso, precisamos de um sistema que permita revisar as regras quando novos exemplos são adicionados.

Duda, R. O., Hart, P. E., and Stork, D. G. (2001). Pattern Classification (2 ed.). John Wiley & Sons, New York

13

Estratégias usadas em design de pipelines



- Pipelines podem ser construídos de diferentes maneiras.
- **Manualmente:** desde o início ou modificando um pipeline existente;
- **Automaticamente:** Através de um planejador ou um modelo de aprendizagem de máquina.

14

Estratégias usadas em design de pipelines



Estratégias manuais:

- **Operadores:**
 - Em uso desde os anos 1970 na área de planejamento de IA, uma subárea de IA (Russel e Norvig, 2016).
 - Objetiva projetar estratégias de sequências de ações que podem ser executadas por um sistema.
 - Originalmente o sistema podia ser um agente inteligente ou um robô.
 - O conceito é adaptado aqui para planejar pipelines.
 - A seleção dos operadores é guiada por pre- e pós-condições:
 - Pré-condições: são os requisitos para operação ser aplicável.
 - Pós-condições: são os efeitos, ou seja, como ficará o estado após a operação.
 - Dessa maneira pode-se concatenar os operadores.
- **Modificação manual de pipelines existentes:**
 - O usuário fornece uma descrição em alto-nível da tarefa alvo.
 - O sistema atua como um tipo de sistema de raciocínio baseado em casos que busca pela tarefa mais similar em experimentos passados.
 - Os pipelines das tarefas similares são então apresentados ao usuário que pode adaptá-lo à tarefa alvo.
 - Esta foi a abordagem usada no projeto MiningMart, cujo foco era pré-processamento.
 - A ideia do MiningMart era permitir o reuso de bons pipelines de pré-processamento em diferentes tarefas por meio de raciocínio baseado em casos.

Russell, S. J. and Norvig, P. (2016). Artificial Intelligence: A Modern Approach. Prentice Hall, 3rd edition

15

Estratégias usadas em design de pipelines



Estratégias automáticas

- **Planejadores:**
 - Muitos sistemas usam um planejador para projetar seus pipelines. A exemplo de:
 - CITRUS (Engels et al., 1997; Wirth et al., 1997);
 - IDA (Bernstein and Provost, 2001);
 - eIDA (Kietz et al., 2012);
 - Auto-WEKA (Thornton et al., 2013; Kotthoff et al., 2016);
 - Auto-sklearn (Feurer et al., 2015).

16

Estratégias usadas em design de pipelines



- O gerador de plano tem como entrada:
 - Um dataset;
 - Um objetivo definido pelo usuário (ex.: construir um classificador);
 - Informações fornecidas pelo usuário sobre os dados;
 - Informações que não podem ser obtidas automaticamente.
- O planejador começa com um processo vazio e sistematicamente busca por uma operação cujas pré-condições estejam satisfeitas e cujos indicadores são congruentes com as preferências definidas pelo usuário.
- Assim que uma operação é encontrada, é adicionada ao processo atual e suas pós-condições tornam-se as novas condições (estado) do sistema, a partir das quais a busca continua.
- A busca finaliza quando o objetivo é satisfeito.
- No sistema IDA a busca realizada pelo planejador é exaustiva, todos os pipelines válidos são retornados.
- Isto funcionava porque o espaço de busca era pequeno. Com espaços de busca maiores, precisamos empregar estratégias que permitam uma busca heurística, não exaustiva no espaço.

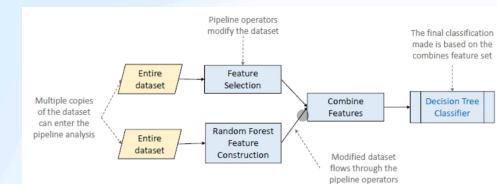
17

Estratégias usadas em design de pipelines



TPOT - Tree-based pipeline optimization tool

- Este método aplica uma busca heurística, usando um algoritmo evolucionário (Genetic Programming) para abordar o problema CASH (Olson et al., 2016).
- Funciona evoluindo vários pequenos pipelines transformando-os em pipelines maiores.
- Os pipelines pequenos iniciais costumam conter um operador de pré-processamento e um único classificador.
- Uma função de cruzamento mescla operadores de dois pipelines. Na figura está representado pelo “Combine Features”.
- Somente um classificador prevalece. Quando o processo é repetido por várias gerações, os pipelines vão ficando maiores e mais complexos.
- Há também um operador de mutação que pode adaptar o pipeline adicionando ou removendo operadores.



Olson, R. S., Bartley, N., Urbanowicz, R. J., and Moore, J. H. (2016). Evaluation of a tree-based pipeline optimization tool for automating data science. In Proceedings of the Genetic and Evolutionary Computation Conference 2016, pages 485–492.

18

Estratégias usadas em design de pipelines



GAMA - General automatic machine learning assistant

- Este método também aplica Genetic Programming para realizar a busca, similarmente ao TPOT.
- No entanto, TPOT usa uma estratégia síncrona, avaliando todos os pipelines candidatos em cada geração, o GAMA é assíncrono, o que frequentemente é mais rápido, já que a avaliação de uma geração não é atrasada por um único ou poucos pipelines lentos.
- GAMA também é modular, permite que o usuário troque a técnica de otimização ou acrescente um pós-processamento (ex.: construir um ensemble com os pipelines gerados).

Gijsbers, P. and Vanschoren, J. (2019). GAMA: Genetic automated machine learning assistant. Journal of Open Source Software, 4(33):1132.

19

Estratégias usadas em design de pipelines



Métodos para redução do espaço de busca

- Várias abordagens de AutoML possuem uma etapa para reduzir o espaço de busca, tornando mais célere a geração de pipelines.
- Nguyen et al. (2014) usa um modelo para guiar o planejador durante o design de pipelines. O meta-modelo é treinado usando uma abordagem de aprendizado por semelhança.
- Auto-sklearn (Feurer et al., 2015), usa meta-aprendizagem para realizar o *warm-start* de uma otimização Bayesiana.
- AlphaD3M (Drori et al. 2018) é um sistema de AutoML que usa (meta) aprendizado por reforço.
- Naive AutoML (Mohr and Wever, 2021) foi pensado para ser um baseline entre os métodos de busca de pipelines, onde cada componente é buscado sob uma suposição de interdependência. Isto diminui drasticamente o espaço de busca.
- Em Maia et al. (2025), um método de curadoria do espaço de busca é proposto, reduzindo a complexidade e melhorando o desempenho.

Nguyen, P., Hilario, M., and Kalousis, A. (2014). Using meta-mining to support data mining workflow planning and optimization. Journal of Artificial Intelligence Research, 51:605–644.

Feurer, M., Springenberg, J., and Hutter, F. (2015b). Initializing Bayesian hyperparameter optimization via meta-learning. In Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, pages 1128–1135.

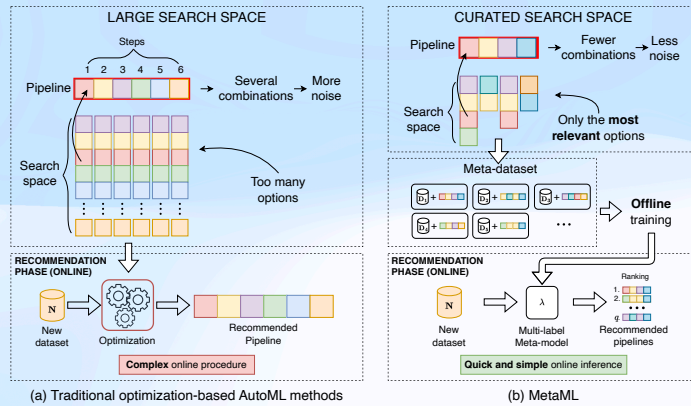
Drori, I. et al. (2018). AlphaD3M: Machine learning pipeline synthesis. In Workshop AutoML 2018 @ ICML/IJCAI-ECAI.

Mohr, F. and Wever, M. (2021). Naive automated machine learning—a late baseline for automl. arXiv preprint arXiv:2103.10496.

Maia, C.M., de Amorim, L.B.V., Cavalcanti, G.D.C., Cruz, R.M.O. (2025). MetaML: a multi-label meta-learning approach for pipeline recommendation. Machine Learning 114.

20

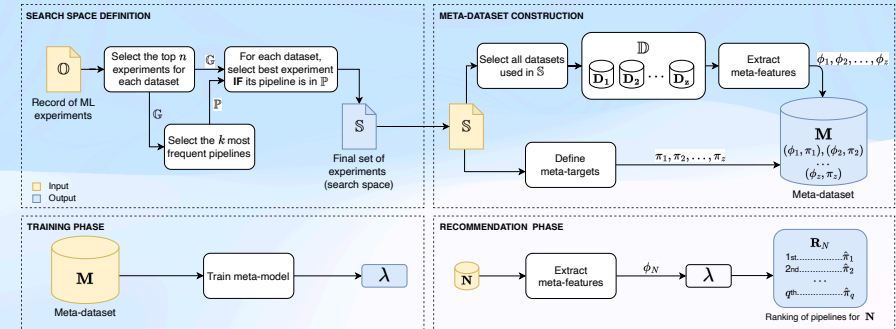
MetaML: A multi-label meta-learning approach for pipeline recommendation



Maia, C.M., de Amorim, L.B.V, Cavalcanti, G.D.C, Cruz, R.M.O. (2025). MetaML: a multi-label meta-learning approach for pipeline recommendation. Machine Learning 114.

21

MetaML: A multi-label meta-learning approach for pipeline recommendation



Maia, C.M., de Amorim, L.B.V, Cavalcanti, G.D.C, Cruz, R.M.O. (2025). MetaML: a multi-label meta-learning approach for pipeline recommendation. Machine Learning 114.

22

MetaML: A multi-label meta-learning approach for pipeline recommendation



Table: Comparison of the most popular open-source AutoML frameworks. FE - Feature Engineering, MS - Model Selection, HO - Hyper-parameters Optimization, DP - Data Preprocessing.

AutoML Framework	Optimization	FE	MS	HO	DP	SS Curation	Meta-Learning	Online Search
Auto-WEKA	SMAC	✓	✓	✓	✓	×	×	✓
AutoSklearn 1.0	SMAC	✓	✓	✓	✓	×	✓	✓
AutoSklearn 2.0	SMAC	✓	✓	✓	✓	×	✓	✓
GAMA	RS,ASHA;AMOE	✓	✓	✓	✓	×	×	✓
Hyperopt-Sklearn	RS;TPE;Annealing	✓	✓	✓	✓	×	×	✓
ML-Plan	PHRT	✓	✓	✓	✓	×	×	✓
Naive AutoML	RS	✓	✓	✓	✓	×	×	✓
TPOT	GP	✓	✓	✓	✓	×	×	✓
TABPFN	Transformer-based	✓	✓	✓	✓	×	×	✓
AutoGluon	Multi-Layer Stack	×	✓	✓	×	×	×	×
FLAML-Zero	Meta-learning only	×	✓	✓	×	×	✓	×
H2O AutoML	RS	×	✓	✓	×	×	×	×
OBOE	CF	×	✓	✓	×	×	×	×
MetaML	Meta-learning only	✓	✓	×	✓	✓	✓	×

Maia, C.M., de Amorim, L.B.V, Cavalcanti, G.D.C, Cruz, R.M.O. (2025). MetaML: a multi-label meta-learning approach for pipeline recommendation. Machine Learning 114.

23

MetaML: A multi-label meta-learning approach for pipeline recommendation



	Mean acc.	Win/tie/loss	Mean rank	p-value	Time (hours)
Auto-WEKA	0.743	76/9/67	6.447	0.594	356
AutoSklearn 1.0	0.774	48/15/89	5.098	0.020	371
AutoSklearn 2.0	0.792	47/13/92	4.171	4.13e-06	361
AutoGluon	0.830	50/8/94	5.457	2.60e-08	13
H2O AutoML	0.789	72/3/77	6.388	0.211	366
Naive AutoML	0.811	50/16/86	4.763	4.13e-05	59
TPOT	0.746	61/13/78	5.447	0.275	357
FLAML-Zero	0.759	91/25/36	7.013	1.48e-08	0.001
TabPFN	0.767	57/13/82	4.223	0.060	23
MetaML Zero-shot	0.798	-	5.990	-	0.056

	Mean acc.	Win/tie/loss	Mean rank	p-value	Time (hours)
Auto-WEKA	0.743	100/11/41	6.611	1.39e-05	356
AutoSklearn 1.0	0.774	70/15/67	5.243	0.303	371
AutoSklearn 2.0	0.792	64/14/74	4.286	0.073	361
AutoGluon	0.830	74/9/69	5.618	0.176	13
H2O AutoML	0.789	92/2/58	6.516	0.017	366
Naive AutoML	0.811	66/22/64	4.888	0.646	59
TPOT	0.746	76/15/61	5.552	0.047	357
FLAML-Zero	0.759	107/23/22	7.111	6.19e-17	0.001
TabPFN	0.767	74/19/59	4.355	0.458	23
MetaML 3-shot	0.821	-	4.815	-	0.108

Maia, C.M., de Amorim, L.B.V, Cavalcanti, G.D.C, Cruz, R.M.O. (2025). MetaML: a multi-label meta-learning approach for pipeline recommendation. Machine Learning 114.

24